

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

01100
10011
10110
01110
01100
10011
10110
10110
10011
10110
10011
1
1
1
0
0
1
1

Los lenguajes orientados a objetos

- Encapsulamiento
- Abstracción
- Herencia
- Polimorfismo

Ligadura dinámica

01100
10011
10110
01110
01100
10011
10110
10110
10011
10110
10011
1
1
1
0
0
1
1

Clasificación

Empleado

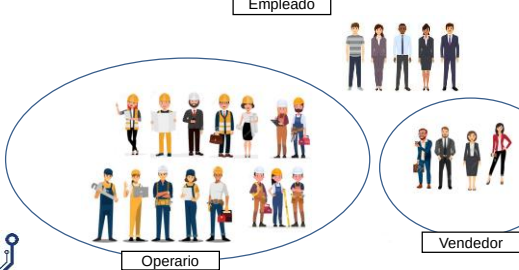


- Consideremos una fábrica en la que trabajan decenas de empleados.
- Cada empleado tiene un número de legajo que lo identifica, una fecha de ingreso y un sueldo básico.
- A partir de la fecha de ingreso se calcula la antigüedad del empleado.
- Algunos de los empleados son operarios y reciben un premio que se computa a partir de las horas trabajadas.
- Los vendedores cobran una comisión que se calcula en base a sus ventas acumuladas.

01100
10011
10110
01110
01100
10011
10110
10110
10011
10110
10011
1
1
1
0
0
1
1

Clasificación

Empleado



Operario

Vendedor

01100
10011
10110
01110
01100
10011
10110
10110
10011
10110
10011
1
1
1
0
0
1
1

Empleado

<<Atributos>>
legajo: entero
sueldoBasico: real
fechaIngreso: Fecha
ventaAcumulada: real
horas: entero

<<Constructor>>
Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerSueldoBasico(sb: real)
acumularVentas(m: real)
inicializarVentas ()
establecerHoras(h: entero)
<<Consultas>>
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntigüedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla, hoy: Fecha): real
obtenerVentaAcumulada(): real
obtenerComisionAnual(): real
obtenerHoras(): entero
obtenerPremio(p: real): real

Podemos tener una única clase con los atributos de los operarios y los vendedores.

¿Está bien eso?

01100
10011
10110
01110
01100
10011
10110
10110
10011
10110
10011
1
1
1
0
0
1
1

Clases que comparten muchas características

Vendedor

<<Atributo de instancia>>
legajo: entero
sueldoBasico: real
fechaIngreso: Fecha
ventaAcumulada: real

<<Constructores>>
Vendedor(leg: entero, sb: real, fi: Fecha, v: real)
<<Comandos>>
establecerSueldoBasico(sb: real)
acumularVentas(m: real)
inicializarVentas ()
<<Consultas>>
obtenerVentaAcumulada(): real
obtenerComisionAnual(): real
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntigüedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

Operario

<<Atributo de instancia>>
legajo: entero
sueldoBasico: real
fechaIngreso: Fecha
horas: entero

<<Constructores>>
Operario(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerSueldoBasico(sb: real)
establecerHoras(h: entero)
<<Consultas>>
obtenerHoras(): entero
obtenerPremio(p: real): real
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntigüedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

La clase Empleado

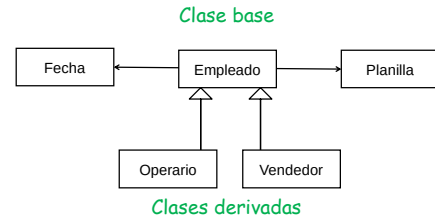
Empleado
<<Atributos>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntiguedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

```

01100
10011
10110
01110
01100
10011
10110
01110
10011
10110
0001
1 11
0 0
1

```

La relación de herencia



```

01100
10011
10110
01110
01100
10011
10110
01110
10011
10110
1001
1 11
0 0
1

```

Las clases Vendedor y Operario son clases derivadas de la clase base Empleado

El concepto de herencia

La herencia jerárquica es un mecanismo que permite organizar clases de acuerdo a relaciones de **generalización-especialización**.

```

01100
10011
10110
01110
01100
10011
10110
01110
10011
10110
0001
1 11
0 0
1

```

La clase Empleado

Empleado
<<Atributos>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntiguedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

obtenerVacaciones(hoy: Fecha): entero
Requiere hoy posterior a fechaIngreso. Retorna la cantidad de días de vacaciones que le corresponden empleado.
7 si antigüedad mayor o igual a 1 y menor o igual a 5.
15 si antigüedad mayor a 5 y menor o igual a 10.
25 si antigüedad es mayor a 10.
Si tiene menos de 1 año de antigüedad, no tiene vacaciones

```

01100
10011
10110
01110
01100
10011
10110
01110
10011
10110
1001
1 11
0 0
1

```

La clase Empleado

Empleado
<<Atributos>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntiguedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

obtenerSueldoNeto(p: Planilla, hoy: Fecha): real
Requiere p ligada y hoy posterior a fechaIngreso
Retorna el sueldo neto del empleado calculado como el sueldo básico incrementado según un porcentaje dependiente de la antigüedad y obtenido de la planilla p.
Si tiene 15 años o más el sueldo neto es el doble del básico

```

01100
10011
10110
01110
01100
10011
10110
01110
10011
10110
1001
1 11
0 0
1

```

La especialización de la clase Empleado

Reusabilidad

Empleado
<<Atributos>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntiguedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

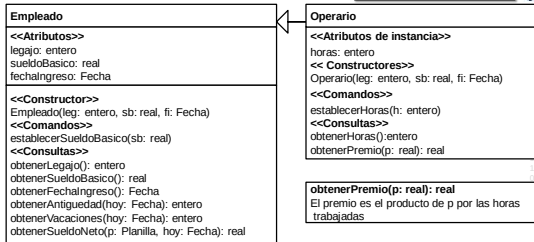
Vendedor
<<Atributo de instancia>> ventaAcumulada: real
<<Constructores>> Vendedor(leg: entero, sb: real, fi: Fecha, v: real)
<<Comandos>> acumularVentas(m: real) inicializarVentas ()
<<Consultas>> obtenerVentaAcumulada(): real obtenerComisionAnual(): real
obtenerComisionAnual(): real La comisión es el 5% de la venta acumulada

Los constructores no se heredan!!!!

Todo objeto de clase Vendedor es un objeto de clase Empleado.

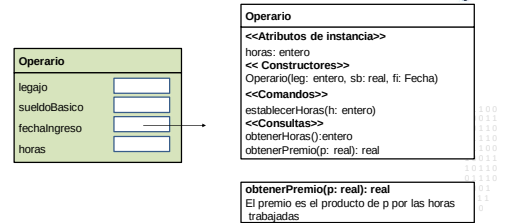
La especialización de la clase Empleado

Reusabilidad



Todo objeto de clase Operario es un objeto de clase Empleado.

La especialización de la clase Empleado



El estado interno de cada instancia de Operario tiene los atributos específicos de su clase y los heredados de la clase Empleado.

Los modificadores de acceso

```
class Empleado {
    protected int legajo;
    protected float sueldoBasico;
    protected Fecha fechaIngreso;
    //Constructor

    //Comandos

    //Consultas
}
```

Los atributos protegidos pueden ser accedidos desde las clases derivadas

Los modificadores de acceso

```
class Empleado {
    protected int legajo;
    protected float sueldoBasico;
    protected Fecha fechaIngreso;
    //Constructor
    public Empleado(int leg, float sb, Fecha fi){
        legajo = leg;
        sueldoBasico = sb;
        fechaIngreso = fi; }
    //Comandos

    //Consultas
}
```

Las clases derivadas de la clase base Empleado heredan atributos y métodos, no los constructores.

El método obtenerVacaciones en Empleado

```
class Empleado {
    ...
    public int obtenerVacaciones(Fecha hoy){
        /*Requiere hoy posterior a fechaIngreso. Retorna la cantidad de dias de
        vacaciones que le corresponden empleado.
        7 si antigüedad mayor o igual a 1 y menor o igual a 5.
        15 si antigüedad mayor a 5 y menor o igual a 10.
        25 si antigüedad es mayor a 10.
        Si tiene menos de 1 año, no tiene vacaciones */
        int a = obtenerAntigüedad(hoy);
        int v = 0;
        if (a > 10) v = 25;
        else
            if (a > 5) v = 15;
            else if (a >= 1) v = 7;
        return v;
    }
    ...
}
```

El método obtenerSueldoNeto en Empleado

```
class Empleado {
    ...
    public float obtenerSueldoNeto(Planilla p, Fecha hoy){
        /*Requiere p ligada y hoy posterior a fechaIngreso
        Retorna el sueldo neto del empleado calculado como el sueldo
        básico incrementado según un porcentaje dependiente de la
        antigüedad y obtenido de la planilla p.
        Si tiene 15 años o más el sueldo neto es el doble del básico*/
        int a = obtenerAntigüedad(hoy);
        float sn = 0;
        int porc;
        if (a >= 15)
            sn = 2 * sueldoBasico;
        else{
            porc = p.obtenerPorcentaje(a);
            sn = sueldoBasico + (sueldoBasico * porc / 100); }
        return sn;
    }
}
```

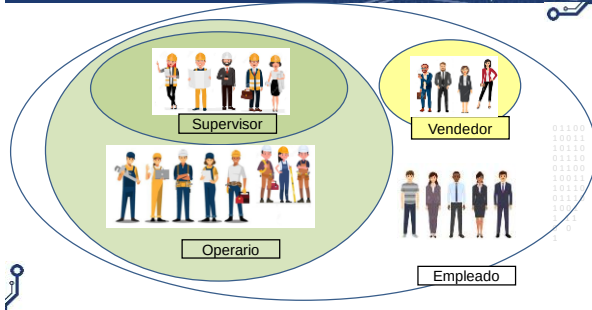
La clase Operario

```
class Vendedor extends Empleado {
    //Atributos de instancia
    protected float ventasAcumuladas;
    //Constructor
    public Vendedor (int leg, float sb, Fecha fi, float v){
        super(leg,sb,fi);
        ventasAcumuladas = v; }
    //Comandos
    public void acumularVentas(float m){
        ventasAcumuladas += m; }
    public void inicializarVentas (){
        ventasAcumuladas = 0; }
    //Consultas
    public float obtenerVentaAcumulada(){
        return ventasAcumuladas; }
    public float obtenerComisionAnual(){
        return ventasAcumuladas * 0.05; }
}
```

La clase Operario

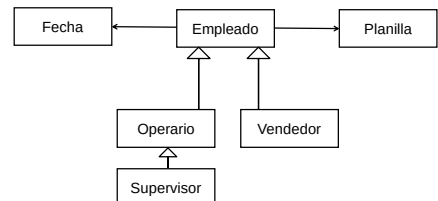
```
class Operario extends Empleado {
    //Atributos de instancia
    protected int horas;
    //Constructor
    public Operario (int leg, float sb, Fecha fi){
        super(leg,sb,fi);
    }
    //Comandos
    public void establecerHoras(int h){
        horas = h; }
    //Consultas
    public int obtenerHoras (){
        return horas; }
    public float obtenerPremio (float p){
        return horas * p ; }
}
```

Clasificación



La relación entre clases

Extensibilidad



Todo objeto de clase Supervisor ES UNA instancia de Operario y también de Empleado.

La clase Empleado

```
Empleado
<<Atributos>>
legajo: entero
sueldoBasico: real
fechalngreso: Fecha
<<Constructor>>
Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerSueldoBasico(sb: real)
<<Consultas>>
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntiguedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla,hoy:Fecha):real
```

```
obtenerVacaciones(hoy: Fecha): entero
Requiere hoy posterior a fechalngreso. Retorna la
cantidad de días de vacaciones que le corresponden
empleado.
7 si antigüedad mayor o igual a 1 y menor o igual a 5.
15 si antigüedad mayor a 5 y menor o igual a 10.
25 si antigüedad es mayor a 10.
Si tiene menos de 1 año, no tiene vacaciones
```

La redefinición

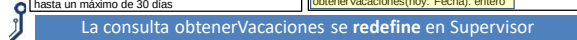
```
Empleado
<<Atributos>>
legajo: entero
sueldoBasico: real
fechalngreso: Fecha
<<Constructor>>
Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerSueldoBasico(sb: real)
<<Consultas>>
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntiguedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla,hoy:Fecha):real

Operario
<<Atributos>>
horas: entero
<<Constructores>>
Operario(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerHoras(h: entero)
<<Consultas>>
obtenerPremio(p: real): real

Supervisor
<<Constructores>>
Supervisor(leg: entero, sb: real, fi: Fecha)
<<Consultas>>
obtenerVacaciones(hoy: Fecha): entero
```

El cambio no afecta a las clases ya implementadas y verificadas

La redefinición



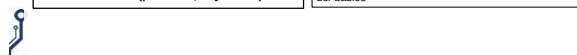
La redefinición

El método obtenerVacaciones en Empleado

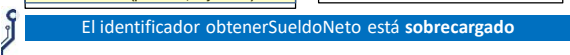
El método obtenerVacaciones en Supervisor

El método `obtenerVacaciones` de la clase `Empleado` queda derogado para las instancia de la clase `Supervisor`.

La clase Empleado



La sobrecarga



La sobrecarga

Cuando dos o más métodos tienen el mismo nombre pero distinto tipo o número de parámetros, el identificador está **sobrecargado**.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
1111
000
1
```

El método obtenerSueldoNeto en Empleado

```
class Empleado {
    public float obtenerSueldoNeto(Planilla p, Fecha hoy){
        /*Requiere p ligada y hoy posterior a fechaIngreso
        Retorna el sueldo neto del empleado calculado como el sueldo
        básico incrementado según un porcentaje dependiente de la
        antigüedad y obtenido de la planilla p.
        Si tiene 15 años o más el sueldo neto es el doble del básico*/
        int a = obtenerAntigüedad(hoy);
        float sn = 0;
        int porc;
        if (a >= 15)
            sn = 2 * sueldoBasico;
        else{
            porc = p.obtenerPorcentaje(a);
            sn = sueldoBasico + (sueldoBasico * porc / 100);
        }
        return sn;
    }
}
```

```
0
1
0
0
0
1
0
0
0
0
```

El método obtenerSueldoNeto en Vendedor

```
class Vendedor extends Empleado {
    //Atributos de instancia
    protected float ventaAcumulada;

    ...

    public float obtenerSueldoNeto(float r){
        /*El sueldo neto es el sueldo básico más la rentabilidad r*/
        return sueldoBasico + r;
    }
}
```

```
0
1
0
0
0
0
0
0
0
0
0
0
0
0
0
```

El método obtenerSueldoNeto en Vendedor

```
class Vendedor extends Empleado {
    //Atributos de instancia
    protected float ventaAcumulada;

    ...

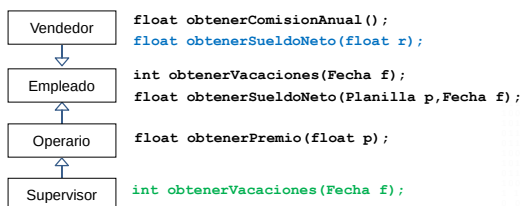
    public float obtenerSueldoNeto(float r){
        /*El sueldo neto es el sueldo básico más la rentabilidad r*/
        return obtenerSueldoBasico() + r;
    }
}
```

```
0
1
0
0
0
0
0
0
0
0
0
0
0
0
0
```

Como los atributos de la clase Empleado están protegidos, son visibles para la clase Vendedor.

Si sueldoBasico estuviera encapsulado (declarado privado) solo podría accederse a través de los servicios públicos.

La sobrecarga y la redefinición

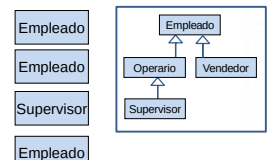


Las clases derivadas heredan los atributos y métodos de la clase Empleado

HERENCIA

```
Empleado e1 = new Empleado(125,21000,f);
Operario e2 = new Operario(126,20000,f);
Supervisor e3 = new Supervisor(120,30000,f);
Vendedor e4 = new Vendedor(129,50000,f,10000);

int c;
c = e1.obtenerVacaciones(h);
c = e2.obtenerVacaciones(h);
c = e3.obtenerVacaciones(h);
c = e4.obtenerVacaciones(h);
```



HERENCIA

```
Empleado e1 = new Empleado(125,21000,f);
Operario e2 = new Operario(126,20000,f);
Supervisor e3 = new Supervisor(120,30000,f);
Vendedor e4 = new Vendedor(129,50000,f,10000);
float c;

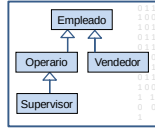
c = e1.obtenerPremio(15);
c = e2.obtenerPremio(15);
c = e3.obtenerPremio(15);
c = e4.obtenerPremio(15);
```

Error

Operario

Operario

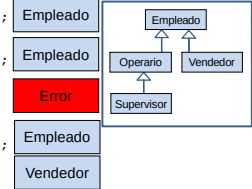
Error



HERENCIA

```
Empleado e1 = new Empleado(125,21000,f);
Operario e2 = new Operario(126,20000,f);
Supervisor e3 = new Supervisor(120,30000,f);
Vendedor e4 = new Vendedor(129,50000,f,10000);
float b1, b2, b3, b4, b5;

b1 = e1.obtenerSueldoNeto(p,h);
b2 = e2.obtenerSueldoNeto(p,h);
b3 = e3.obtenerSueldoNeto(r);
b4 = e4.obtenerSueldoNeto(p,h);
b5 = e4.obtenerSueldoNeto(r);
```



El concepto de polimorfismo

El **polimorfismo** en Ciencias de la Computación es el mecanismo que permite que un mismo nombre esté asociado a distintas abstracciones.

El polimorfismo en Java

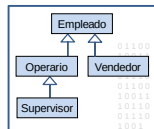
Una **variable polimórfica** puede quedar asociada a objetos de diferentes clases.

Una **asignación polimórfica** liga un objeto de una clase a una variable declarada de otra clase.

Un **método polimórfico** incluye una o más variables polimórficas como parámetro.

Las variables polimórficas

```
Empleado e1, e2, e3, e4;
```

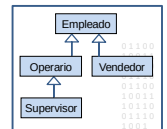


Las variables e1, e2, e3 y e4 son polimórficas, pueden estar ligadas a objetos de clase Empleado o cualquiera de sus clases derivadas

Las asignaciones polimórficas

```
Empleado e1, e2, e3, e4;
```

```
e1 = new Empleado(125,21000,f);
e2 = new Operario(126,20000,f);
e3 = new Supervisor(120,30000,f);
e4 = e3;
```



La ligadura dinámica



La relación entre clases

Colección polimórfica



Colección polimórfica

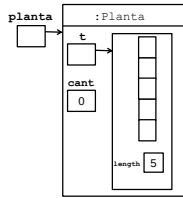


La relación entre clases

Colección polimórfica



Colección polimórfica



```
Planta planta = new Planta(5);
```

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
1111
0 0
```

Colección polimórfica

```
public void alta (Empleado e){
    /*asigna el empleado e a la primera posición libre de la estructura.
    Aumenta el valor de cant. Requiere que la clase cliente haya
    verificado que la estructura no esté llena, e esté ligado y no haya un
    empleado con el mismo número de legajo*/
    t[cant] = e;
    cant++;
}
```

```
0
1
0
0
0
0
1
0
0
0
0
```

El comando alta es un método polimórfico, el parámetro real puede ser de clase Empleado o de cualquiera de sus clases descendientes.

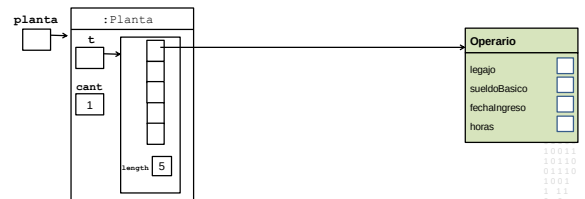
Colección polimórfica

```
public void alta (Empleado e){
    /*asigna el empleado e a la primera posición libre de la estructura.
    Aumenta el valor de cant. Requiere que la clase cliente haya
    verificado que la estructura no esté llena, e esté ligado y no haya un
    empleado con el mismo número de legajo*/
    t[cant] = e;
    cant++;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
1111
0 0
```

```
Planta planta = new Planta(5);
Operario o = new Operario(345,15000,£);
planta.alta(o);
```

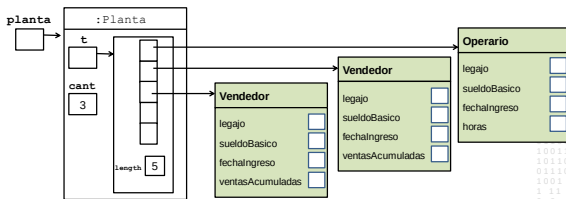
Colección polimórfica



```
Planta planta = new Planta(5);
Operario o = new Operario(345,15000,£);
planta.alta(o);
```

```
01100
10110
01110
10011
1111
0 0
```

Colección polimórfica



```
Planta planta = new Planta(5);
Operario o = new Operario(345,15000,£);
planta.alta(o);
planta.alta(new Vendedor(346,20000,£,1000000));
planta.alta(new Vendedor(347,25000,£,50000));
```

```
01100
10110
01110
10011
1111
0 0
```

Colección polimórfica

```
public void alta (Empleado e){
    /*asigna el empleado e a la primera posición libre de la estructura.
    Aumenta el valor de cant. Requiere que la clase cliente haya
    verificado que la estructura no esté llena, e esté ligado y no haya un
    empleado con el mismo número de legajo*/
    t[cant] = e;
    cant++;
}
```

```
01100
00111
01110
11110
00111
11100
01110
01110
11110
0011
0 0
```

La clase Planta es polimórfica porque define métodos polimórficos.

```
Planta planta = new Planta(5);
Operario o = new Operario(345,15000,£);
planta.alta(o);
planta.alta(new Vendedor(346,20000,£,1000000));
planta.alta(new Vendedor(347,25000,£,50000));
```

